

Universidad Nacional de Avellaneda



Ingeniería en Informática

Trabajo Práctico N°2

Sistema de Control - Lazo Cerrado

Alumnos

Juan David

Karol Peña

Iván Kwist

Índice de Contenido

Objetivo	2
Introducción	2
Desarrollo	
Diagrama en Bloques y Ecuaciones	3-4
Implementación	5-10
Medición	11-12
Simulaciones	13-14
Conclusiones	15
Anexos	
Anexo A	16

Objetivo

El objetivo de este segundo informe es el de explicar cómo fue desarrollada la etapa de control del nuestro sistema. Se describirá tanto la parte circuital como la de software, con el objetivo de observar cómo aplica la teoría de diseño de sistemas de control a nuestra implementación.

Introducción

Como ya hemos mencionado, en esta segunda parte del trabajo práctico vamos a desarrollar la etapa de control de temperatura de nuestra heladera. Para ello, vamos a sensar la temperatura del interior de la misma, procesarla, y generar una señal de PWM (Modulación de Ancho de Pulso) que controlará la Celda de Peltier de la heladera. Estas tres acciones serán llevadas a cabo por un microcontrolador, que actuará bajo las órdenes de un programa de nuestra autoría.

Desarrollo

Diagrama en Bloques y Ecuaciones

El diagrama en bloques del sistema de control planteado es el siguiente:

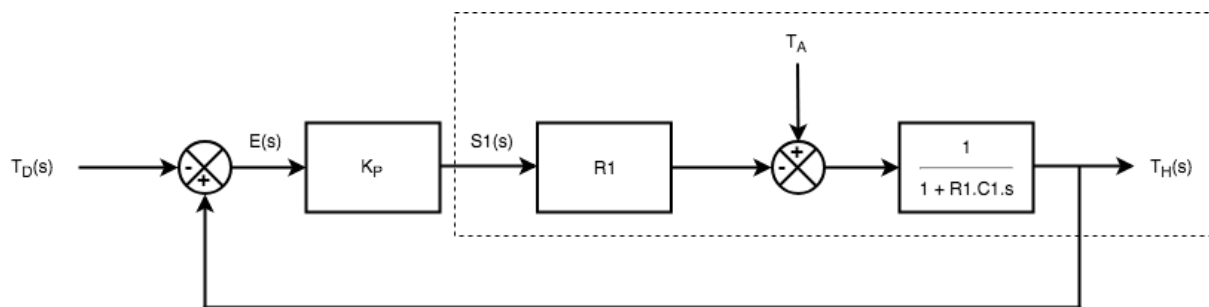


Figura 1 - Diagrama en Bloques

En el cual $T_D(s)$ es la señal de referencia (o temperatura deseada), $E(s)$ es la señal de error (o diferencia entre la temperatura medida y la temperatura deseada) y K_P es una constante proporcional de diseño. En el recuadro de línea punteada podemos ver el diagrama en bloques con el que veníamos trabajando.

La constante K_P es lo que se llama ganancia proporcional. En un controlador proporcional la señal de error se multiplica por ella para obtener su salida. En nuestro caso, su valor determina la potencia que entregará a la entrada de la heladera por cada grado centígrado de diferencia entre la temperatura deseada y la temperatura medida.

Para calcularla fue primero necesario determinar en qué rango de temperaturas nuestro sistema iba a ser lineal. Elegimos un rango de 5°C . Es decir, si la diferencia de temperatura entre la referencia y la medición es de más de 5°C , el microcontrolador entregará la máxima potencia. De la misma manera, lógicamente, si la temperatura medida es menor a la de referencia, el microcontrolador no entregará potencia a la entrada de la heladera.

De manera que el valor de K_P se obtiene como el cociente entre la potencia de entrada máxima y el rango de operación.

$$K_P = \frac{P_{MAX}}{^{\circ}\text{C}} = \frac{12,13 \text{ W}}{5^{\circ}\text{C}}$$

$$K_P \approx 2,4 \text{ W}/^{\circ}\text{C}$$

Podemos observar en el siguiente gráfico el valor de la potencia de entrada en función de la diferencia de temperaturas.

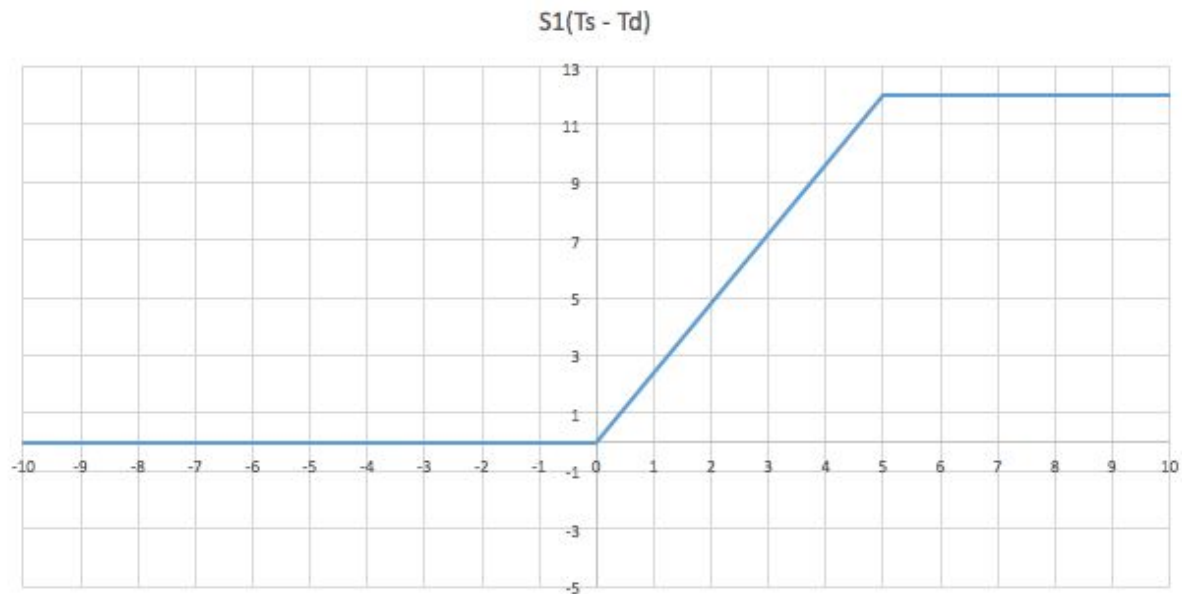


Figura 2 - Potencia de entrada en función de la diferencia de temperatura

Matemáticamente, la expresión de la potencia de entrada en función de la diferencia de temperatura resulta:

$$S1(Ts - Td) = \begin{cases} 0 & (Ts - Td) < 0 \\ K_p (Ts - Td) & 0 \leq (Ts - Td) < 5 \\ 12.13 & (Ts - Td) \geq 5 \end{cases}$$

Por lo tanto, como podemos observar, el sistema se comporta de manera lineal solo en para un intervalo de temperaturas (de 15°C a 20°C).

Del diagrama en bloques obtenemos la siguiente ecuación para la temperatura medida:

$$T_H(s) = \frac{K_p \cdot R1}{1 + K_p \cdot R1 + R1 \cdot C1 s} \cdot T_D(s) + \frac{1}{1 + K_p \cdot R1 + R1 \cdot C1 s} \cdot T_A$$

Este cálculo puede encontrarse en el Anexo A.

Implementación

Vamos a separar la implementación del sistema en dos partes: hardware y software. La parte del hardware está compuesta por la heladera en si, el microcontrolador, los sensores y el circuito intermedio para conectar estos dos componentes.

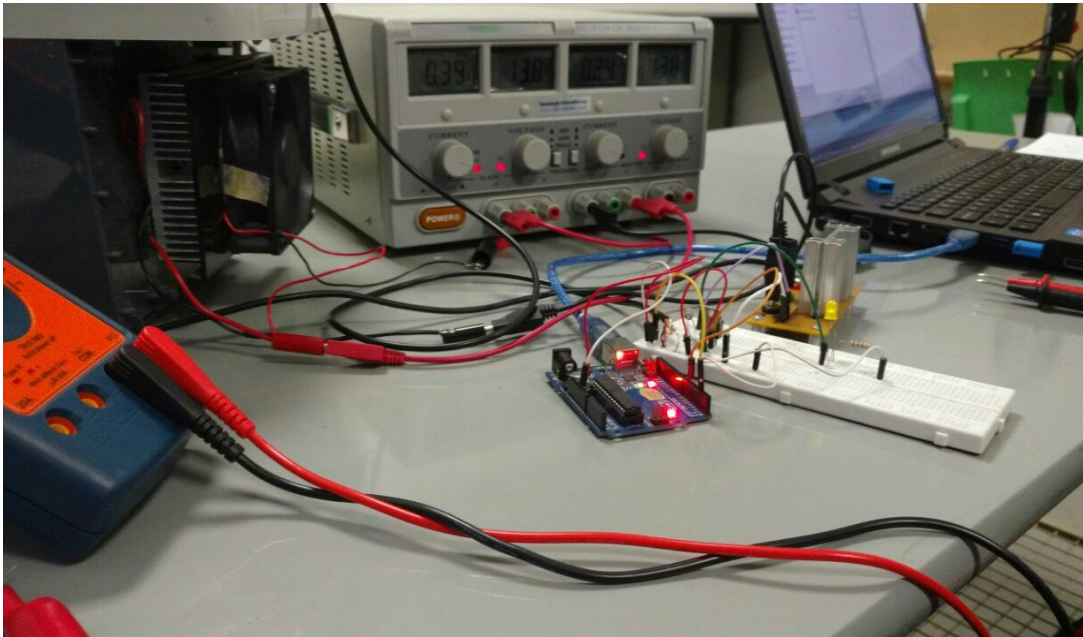


Figura 3 - Implementación del Sistema

Hardware

Sensor de Temperatura

Para medir la temperatura de salida fue necesario utilizar un sensor. El elegido fue el DS18B20, de Maxim IC. Se comunica por el protocolo 1-Wire, que permite una comunicación de conexión simple donde la tanto la alimentación como los datos viajan por el mismo cable. Posee una resolución configurable de 9 a 12 bits y opera en un rango de -55 a 125 grados centígrados.



Figura 4 - Sensor DS18B20

Microcontrolador

Arduino Uno es un microcontrolador basado en el chip ATmega328P. Posee 13 pines digitales de entrada/salida (seis de los cuales pueden ser utilizados de salida pwm), 6 entradas analógicas, un cristal de 16 MHz y una conexión USB. Elegimos este microcontrolador por la facilidad que implica programarlo. Tan solo fue necesario conectarlo a una computadora por su puerto USB y cargar el programa mediante el Arduino IDE, más desarrollado en la sección de Software.



Figura 5 - Microcontrolador Arduino Uno

Circuito

El circuito utilizado para controlar la heladera con el microcontrolador es el siguiente:

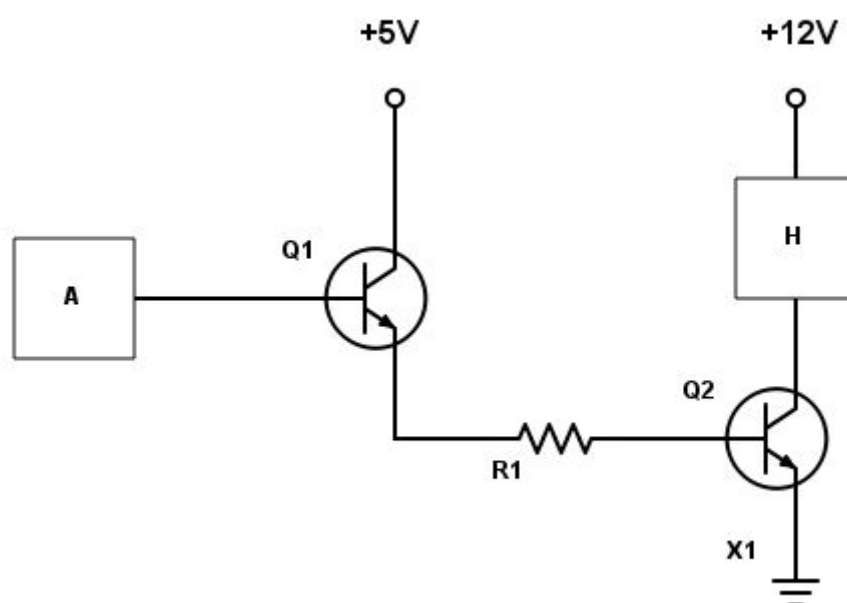


Figura 6 - Circuito del Sistema

Donde A representa al microcontrolador y H a la heladera.

Sin entrar en mucho detalle, $Q1$ y $Q2$ son transistores TJB. La función de $Q1$ es traducir una corriente pequeña en una mayor. Esto hizo falta, ya que la corriente de salida de la placa Arduino Uno (aproximadamente 15 mA) no era suficiente como para activar el transistor $Q2$. La función del transistor $Q2$ es la de permitir cerrar el circuito de la heladera con una pequeña señal de control de $5V$. Esto hizo falta ya que Arduino trabaja con una tensión máxima de $5V$ y la heladera precisa $12V$.

Software

A continuación, se explicará el código utilizado para el control de la temperatura de la heladera. Para esto analizaremos cada sección del mismo.

Encabezado y setup:

```
#include <OneWire.h>
#include <DallasTemperature.h>
#define ONE_WIRE_BUS 2
#define PWM_PIN 6

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);
unsigned long tiempoInicial = 0;

void setup(void) {
    pinMode(PWM_PIN, OUTPUT);
    tiempoInicial = millis();

    Serial.begin(9600);
    Serial.println("Sistema de Control de Temperatura");
    sensors.begin();
}
```

En principio, se incluyen las librerías necesarias para trabajar con el tipo de sensor mencionado, el cual usa protocolo de comunicación 1-Wire. Esto nos permitirá luego tomar las mediciones del mismo y operar en base a ellas. Se asigna el pin que se utilizará para el sensor.

En la función setup se configura el pin que realizará la acción de control, se define el tiempo inicial en milisegundos con la función millis (el momento en el que el programa comienza a ejecutarse), se configura la velocidad de transmisión en bits y se inicializa el sensor.

Loop:

```
void loop(void) {  
    int valorPWM = 0;  
    float tDeseada = 15, tMedida = 0, error = 0;  
    unsigned long deltaTiempo = 0, tiempoActual = 0;  
  
    tiempoActual = millis();  
    sensors.requestTemperatures();  
  
    tMedida = sensors.getTempCByIndex(0);  
  
    Serial.print(tMedida);  
    Serial.print("\t");  
  
    error = -(tDeseada - tMedida);  
    valorPWM = actuador(error);  
  
    deltaTiempo = tiempoActual - tiempoInicial;  
  
    Serial.println(deltaTiempo);  
  
    delay(1000);  
}
```

Se declaran las variables referentes a la temperatura deseada y la medida, el error (definido como la diferencia entre ambas). Con respecto al tiempo, deltaTiempo se define como la diferencia entre el tiempo actual de cada muestra de temperatura y el tiempo en el cual inició el programa. Dichas muestras se toman cada 1 segundo, lo cual está dado por la función delay, que recibe 1000 milisegundos como parámetro.

Por cada ciclo del loop se toma una muestra de temperatura de la heladera a través del sensor y se imprime por pantalla, se calcula el error de temperatura y se llama a la función actuador (explicada en la sección siguiente), pasándole dicho error como parámetro. El valor que devuelve esa función es lo que define la potencia de entrada que se entregará por el pin PWM y es en la que se basa la acción de control del sistema.

Por último, se calcula el deltaTiempo, se imprime en pantalla y se espera 1 segundo para volver a comenzar el loop.

Actuador:

Como se mencionó antes, esta función se encarga de calcular la potencia de entrada que se entregará a la heladera de manera proporcional.

```
int actuador(float errorTemperatura) {
    int valorPWM = 0;

    Serial.print(errorTemperatura);
    Serial.print("\t");

    if (errorTemperatura>0 and errorTemperatura<=5) {
        //Conversion a valor PWM
        valorPWM = (int) (51 * errorTemperatura);
        Serial.print(valorPWM);
        Serial.print("\t");
    }
    else if (errorTemperatura>5) {
        valorPWM = 255;
        Serial.print(valorPWM);
        Serial.print("\t");
    }
    else{
        valorPWM = 0;
        Serial.print(valorPWM);
        Serial.print("\t");
    }

    //Activador PWM
    analogWrite(PWM_PIN, valorPWM);
}
```

Al recibir el error de temperatura (diferencia entre lo medido y lo deseado), se imprime dicho valor. Luego, dependiendo el valor del mismo, se aplica una potencia nula, total o proporción de ésta. Para esto, primero se calcula el valor PWM para cada caso. Recordando la función definida previamente para K_p , la cual es:

$$S1(T_s - T_d) = \begin{cases} 0 & (T_s - T_d) < 0 \\ K_p (T_s - T_d) & 0 \leq (T_s - T_d) < 5 \\ 12.13 & (T_s - T_d) \geq 5 \end{cases}$$

- Error ≤ 0 : Ya conseguimos el valor deseado o incluso uno menor, por lo cual no se aplica potencia
- Error > 5 : está por encima del rango de operación en el cual definimos previamente que la respuesta del sistema era lineal, por lo cual no aplicamos proporcionalidad. Se aplica toda la potencia.
- $0 < \text{Error} < 5$: Se aplica una proporción de la potencia total.

Se define que el valor de PWM se encuentra entre 0 y 255. Esto se debe a que la función `analogWrite()`, la cual utilizamos para definir el ciclo de trabajo para el pin PWM en cada caso, se mueve en dicho rango. De esta forma, 0 significa “siempre encendido” y 255 “siempre apagado”.

Sabiendo esto, podemos calcular el valor de nuestra constante proporcional a K_p en función de dicho rango:

$$k = \frac{\text{DutyCycleMax}}{^{\circ}\text{C}} = \frac{255}{5^{\circ}\text{C}}$$

$$k = 51/^{\circ}\text{C}$$

De esta forma, por cada grado centígrado de error, habrá que aumentar el valor PWM en 51 (siempre refiriéndonos a un error entre los 0 y 5 $^{\circ}\text{C}$), aumentando de esta forma el ciclo de trabajo de la señal de entrada.

Una vez obtenido el valor PWM correspondiente, se aplica el mismo mediante la función `AnalogWrite()` al pin configurado previamente.

Medición

El sistema mostrado en la sección de implementación fue puesto a prueba, y obtuvimos el siguiente resultado:

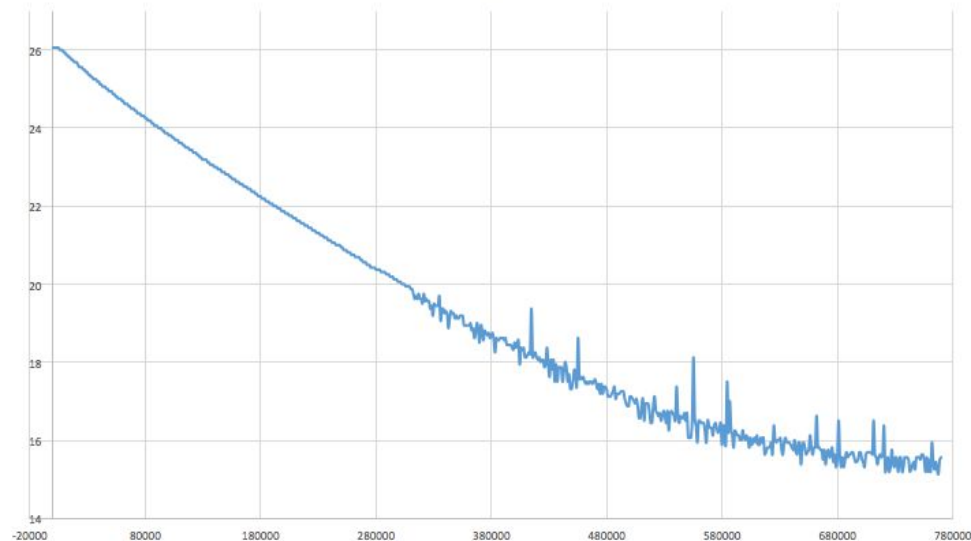


Figura 7 - $T_H(t)$, Temperatura de salida en función del tiempo

Como podemos observar, a partir de la marca de 300000 ms (aproximadamente cinco minutos) la señal presenta ruido y una cierta imprecisión. Esto no es problema del sistema en sí, sino de conexión y comunicación entre el sensor y el microcontrolador. De todos modos, utilizando herramientas gráficas, podemos crear una curva (de línea roja punteada) que muestre la forma que debería mostrar la señal.

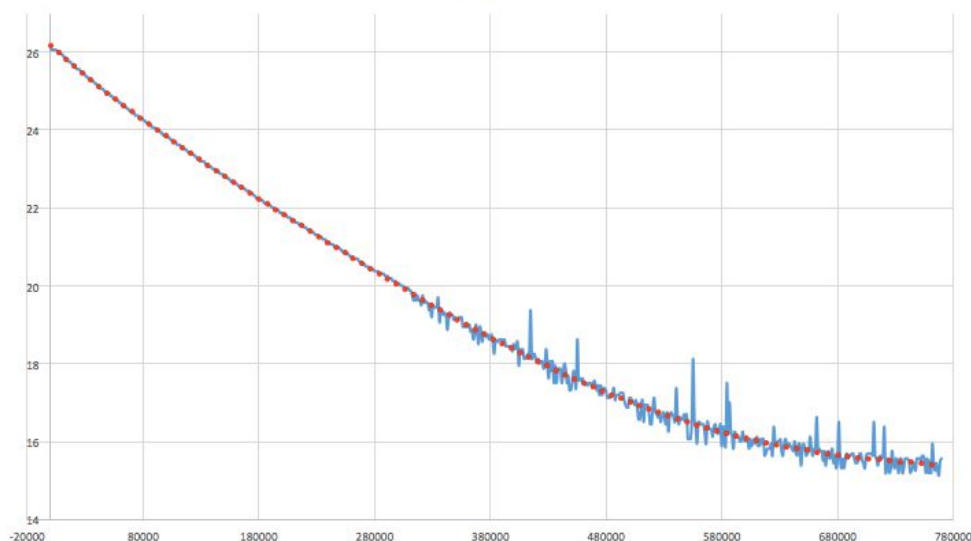


Figura 8 - $T_H(t)$ en celeste y curva de tendencia en rojo

Nuevamente, al igual que en el trabajo anterior, obtenemos algo aproximado a una curva de un sistema de primer orden.

Algo a tener en cuenta es que no llegamos a obtener a la salida la temperatura de referencia ingresada en el programa, sino que obtuvimos una temperatura de aproximadamente $15,5^{\circ}\text{C}$. Medio grado centígrado más de lo que esperábamos. ¿A qué se debe este error?

Echemos un vistazo a la ecuación de la temperatura de salida en el dominio de la frecuencia:

$$T_H(s) = \frac{K_p \cdot R1}{1 + K_p \cdot R1 + R1 \cdot C1 s} \cdot T_D(s) + \frac{1}{1 + K_p \cdot R1 + R1 \cdot C1 s} \cdot T_A$$

Si aplicamos el teorema del valor final obtenemos:

$$\lim_{t \rightarrow \infty} T_H(t) = \frac{K_p \cdot R1}{1 + K_p \cdot R1} \cdot T_D + \frac{1}{1 + K_p \cdot R1} \cdot T_A$$

Es fácil observar que no vamos a obtener exactamente la temperatura deseada T_D a la salida, sino un valor distinto que depende tanto de K_p como de $R1$. $R1$ es una constante en el sistema, mientras que K_p es una variable de diseño. Observemos que a un valor infinitamente grande de K_p , el 1 del denominador del primer término se vuelve despreciable y, además, la influencia de T_A sería nula. En ese caso, obtendríamos la temperatura deseada la salida del sistema.

Llegando a esta conclusión uno creería que lo conveniente sería elegir un valor gigantesco para K_p . Sin embargo, en la realidad hay varias limitaciones que lo impiden.

Simulaciones

El diagrama en bloques de la *Figura 1* fue ingresado en el módulo Xcos del programa matemático Scilab de la siguiente manera:

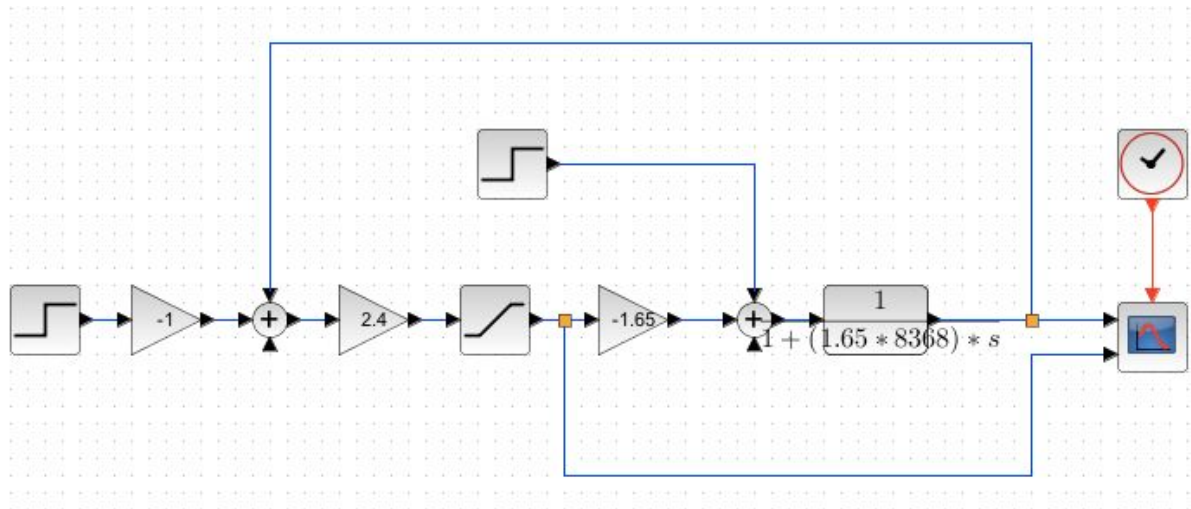


Figura 9 - Esquema del Sistema diagramado en Scilab

Un bloque particular al cual prestarle atención es el de saturación:

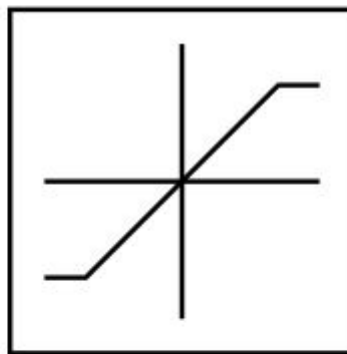


Figura 10 - Bloque de Saturación

Su salida es similar a la señal que aparece en el bloque. Replica la señal de entrada, pero sólo en un determinado intervalo de valores. De esta manera, pudimos representar más fielmente nuestra implementación de K_p , que no era lineal para todo el espectro de temperaturas.

Simulando con una entrada de 15°C (al igual que en nuestra medición), obtuvimos la siguiente salida:

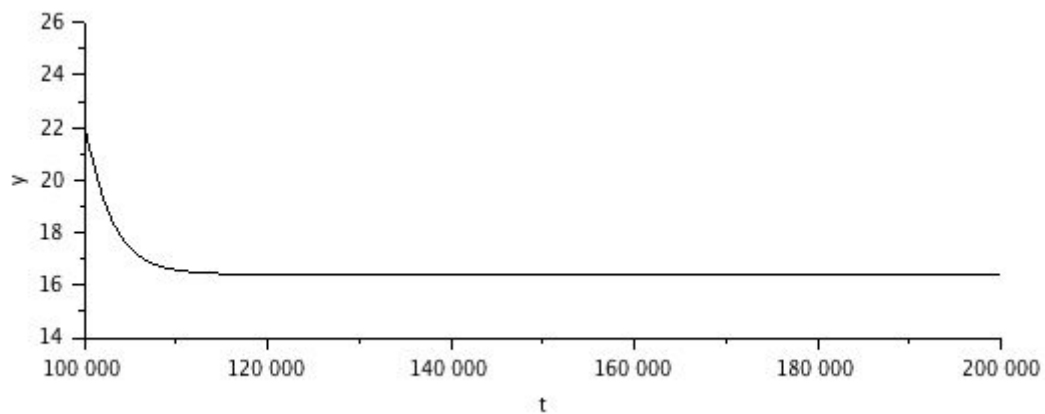


Figura 11 - $T_H(t)$ en negro

El resultado es muy similar a nuestra medición. De hecho, también encontramos un error a la salida.

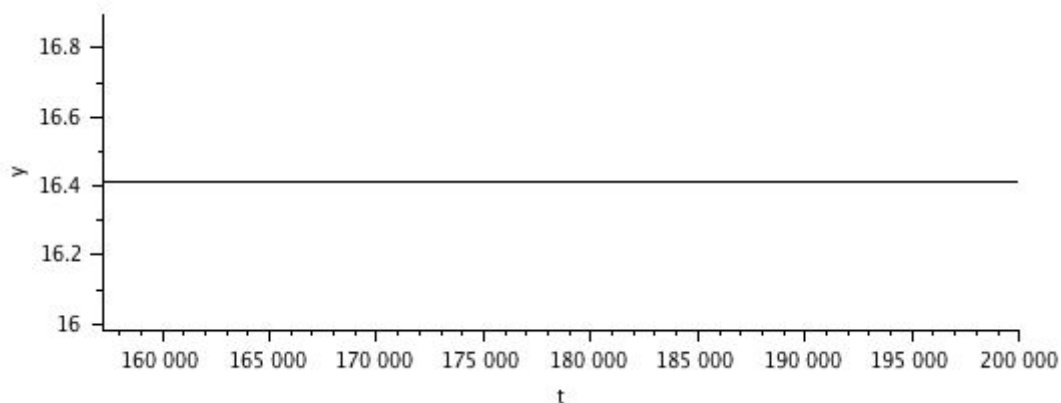


Figura 12 - Detalle de $T_H(t)$

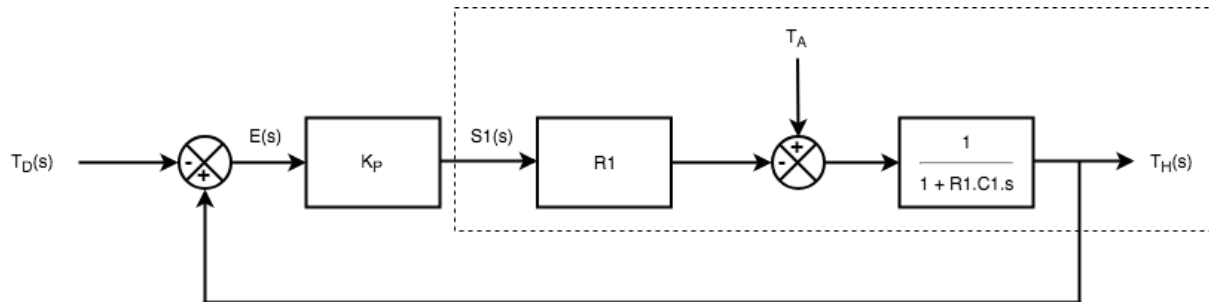
La temperatura de salida se estabiliza en, aproximadamente, 16.4°C . Eso son 0.9°C más de lo que obtuvimos en nuestra medición. Esta diferencia puede deberse a que nuestro modelo matemático no represente de la manera más exacta el modelo real. Puede suceder, por ejemplo, que la potencia máxima de la celda no sea 12.13 W , que la resistencia térmica no sea realmente $1.65\text{ W}/^{\circ}\text{C}$ o que, simplemente, en el momento de la medición la temperatura ambiente no fuera 22°C (temperatura ambiente utilizada en la simulación).

Conclusiones

Al igual que en el trabajo anterior, durante este trabajo vimos cómo modelizar puede ser de gran ayuda a la hora de diseñar un sistema de control. A su vez, vimos cómo la elección de una determinada acción de control puede influenciar la salida de nuestro sistema, aumentando o disminuyendo el error.

Por último, tuvimos la oportunidad de utilizar conocimientos previos de matemática, electrónica y de sistemas embebidos en conjunto con teoría de sistemas de control automático para crear nuestro pequeño sistema.

Anexo A - $T_H(t)$ a partir del Diagrama en Bloques



La mejor manera de resolver esta ecuación es la de aplicar superposición. Primero vamos a calcular la salida para T_A nula y luego para $T_D(s)$ nula.

Si $T_A = 0$:

$$\begin{aligned}
 T_H(s) &= -E(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) &= -(-T_D(s) + T_H(s)) \cdot K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) &= (T_D(s) - T_H(s)) \cdot K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) &= T_D(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} - T_H(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) + T_H(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} &= T_D(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) \cdot (1 + K_P R1 \cdot \frac{1}{1+R1.C1.s}) &= T_D(s) K_P R1 \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) (1 + \frac{K_P.R1}{1+R1.C1.s}) &= T_D(s) \frac{K_P.R1}{1+R1.C1.s} \\
 T_H(s) (1 + \frac{K_P.R1}{1+R1.C1.s}) (1+R1.C1.s) &= T_D(s) K_P R1 \\
 T_H(s) (1+R1.C1.s + K_P.R1) &= T_D(s) K_P R1 \\
 T_H(s) &= T_D(s) \cdot \frac{K_P.R1}{1+R1.C1.s + K_P.R1}
 \end{aligned}$$

Si $T_D(s) = 0$:

$$\begin{aligned}
 T_H(s) &= (-T_H(s) K_P R1 + T_A) \cdot \frac{1}{1+R1.C1.s} \\
 T_H(s) (1 + R1.C1.s) &= -T_H(s) K_P R1 + T_A \\
 T_H(s) (1 + R1.C1.s) + T_H(s) K_P R1 &= T_A \\
 T_H(s) \cdot (1 + R1.C1.s + K_P.R1) &= T_A \\
 T_H(s) &= T_A \cdot \frac{1}{1+R1.C1.s + K_P.R1}
 \end{aligned}$$

Por lo tanto, aplicando superposición obtenemos:

$$T_H(s) = T_D(s) \cdot \frac{K_P.R1}{1+R1.C1.s + K_P.R1} + T_A \cdot \frac{1}{1+R1.C1.s + K_P.R1}$$